
Softwareentwicklung und Programmierung

Grundlagen der Programmierung

1

Organisatorisches - Vorlesung

- Es wird jede Woche eine Vorlesung geben.
 - Diese hat Veranstaltung 5 Leistungspunkte (Credit-Points [CP]).
 - Ein CP steht für ungefähr 10 Arbeitsstunden für qualifizierte Studierende.
 - Planen Sie also mit 150 Arbeitsstunden für die Vorlesung.
 - Aufteilung:
 - Vorlesungs-/ Übungszeit jede Woche 3 h = 15 Wochen * 3 h = 45 h
 - Nacharbeit / Übungsbearbeitung je Woche 5 h = 15 W * 5 h = 75 h
 - Klausurvorbereitung und Durchführung 30 h
 - **bleiben Sie kontinuierlich dabei!**
-

2

Organisatorisches - Klausur

- Klausur
 - Die abschließende Prüfungsleistung wird keine klassische Klausur sondern ein „Kurzbeleg“ sein.
 - Es sind dabei mehrere größere Aufgaben zu bearbeiten.
 - Sie haben für die Bearbeitung einen Zeitraum von 4 Stunden zur Verfügung.
 - Während des gesamten Zeitraumes steht Ihnen eine Konsultationsmöglichkeit zur Verfügung.
 - Die Ergebnisse werden bei der Bewertung verglichen und offensichtliche Täuschungsversuche für alle Beteiligten automatisch mit "durchgefallen" bewertet.
 - **Die Übungsaufgaben sind die beste Vorbereitung!**
-

3

Organisatorisches - Übungen

- Es wird jede Woche ein Übungsblatt geben.
 - **Sie können / sollen die Übungsblätter gemeinsam bearbeiten!**
 - Die Ergebnisse können von jedem Studierenden einzeln, aber unter Angabe der Namen aller Beteiligten, abgegeben werden!
 - Bonuspunkte für Abgabe von Übungsblättern
 - Bewertet werden:
 - Abgabe der (ernsthaft bearbeiteten) Übungen
 - max. eine fehlt = 3 Punkte
 - zwei fehlen = 2 Punkte
 - drei fehlen = 1 Punkt
 - ansonsten = 0 Punkte
 - Bewertung zweier zufällig ausgewählter Übungsblätter mit jeweils einem Punkt. Haben Sie das entsprechende Übungsblatt nicht abgegeben, können Sie auch keinen Punkt dafür bekommen.
 - 100 Klausurpunkte + (maximal) 5 Bonuspunkte = 105 mögliche Punkte
100 Punkte = 1,0, d.h. auch ohne Bonuspunkte erreichbar
-

4

Organisatorisches - Tutorium

- Für diese Veranstaltung gibt es ein optionales Tutorium.
- Ab Vorlesungswoche 2 gibt es pro Woche zwei Termine.
- Wofür?
 - für Nachfragen zur Vorlesung
 - für weitergehende Fragen zu den Themen der Veranstaltung
 - für Hilfe bei der Bearbeitung der Übungsaufgaben
 - für zusätzliche / vertiefende Übungsaufgaben
- **Sie brauchen sich für das Tutorium nicht an- oder abmelden.**

Kommen Sie einfach vorbei.

5

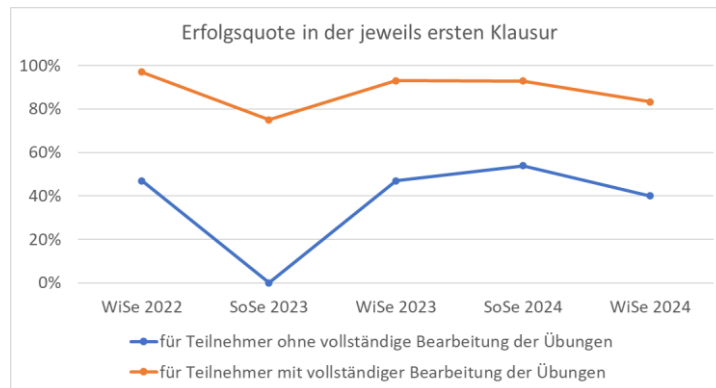
Diese VL wird für Sie dann erfolgreich, wenn Sie ...

- die allgemeinen Prinzipien und die Syntax der Programmiersprache kennen.
- wissen, dass Anfänger sehr lange für das Schreiben von Programmen benötigen.
- selbstständig versuchen, Computerprogramme zu entwerfen und zu schreiben.
- Durchhaltewillen zeigen und nicht nach wenigen Minuten aufgeben.
- alle Programme anhand von einigen Testbeispielen überprüfen.
- wissen, dass die Fehlersuche manchmal sehr lange dauert.
- kontinuierlich die VL nacharbeiten und die Übungen bearbeiten.

(Vgl. S. Dörn, Java lernen in abgeschlossenen Lerneinheiten, Springer Nature 2023)

6

Wie bestehe ich die Klausur?



7

Empfohlene Ausstattung

- **Für alle Studierenden der Wirtschaftsinformatik ist ein Notebook erforderlich.**
- Empfohlene Spezifikationen:
 - Prozessor mit 4 oder 6 Kernen (Windows 11 - fähig)
 - 16 GB Arbeitsspeicher
 - 512 GB SSD (Datenspeicher)
 - Display max. 15,6 Zoll (meine Empfehlung: ≤ 14 Zoll)
 - Windows 10/11 oder Linux
- Wenn Sie ein Apple-Gerät nutzen wollen, beachten Sie bitte, dass die Softwareinstallation oftmals aufwändiger und vielleicht sogar problematisch sein kann.
- Gute Tasche / Rucksack

8

Motto

“Computer science education cannot make anybody an expert programmer any more than studying brushes and pigment can make somebody an expert painter.”

- Eric S. Raymond.

Übersetzung:

Das Studium der Informatik kann niemanden zu einem erfahrenen Programmierer machen, genauso wenig wie das Studium von Pinseln und Pigmenten jemanden zu einem erfahrenen Maler machen kann.

Motto

“Computer science education cannot make anybody an expert programmer any more than studying brushes and pigment can make somebody an expert painter.”

- Eric S. Raymond.

Aber jeder kann die Grundlagen der Programmierung erlernen.

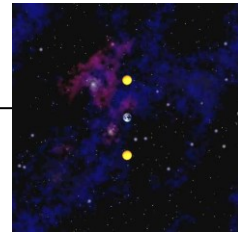
Übersetzung:

Das Studium der Informatik kann niemanden zu einem erfahrenen Programmierer machen, genauso wenig wie das Studium von Pinseln und Pigmenten jemanden zu einem erfahrenen Maler machen kann.

Warum nutzen wir Computer?

Simple Antwort: Weil wir wollen, dass der Computer etwas für uns eine Aufgabe erfüllt.

"Bitte simuliere die Bewegung von N Himmelskörpern unter der Berücksichtigung von Newtons Gesetzen der Bewegung und der Schwerkraft."



Fertige Software: Ist großartig, kann genau das, was der Hersteller möchte.



Warum also Programmieren? Ermöglicht es, einen Computer dazu zu bringen, alles zu tun, was **Sie** wollen!

Nun ja, fast alles...



Ada Lovelace
1843

11

Programmieren. Aber wie?

- **Maschinensprache / Assembler:**
Mühsam und fehleranfällig.

```
10: 8A00  RA ← mem[00]
11: 8B01  RB ← mem[01]
12: 1CAB  RC ← RA + RB
13: 9C02  mem[02] ← RC
14: 0000  halt
```

- **Natürliche Sprachen:**
Mehrdeutig und für Computer oftmals schwer zu verstehen. (Trotz Alexa...)

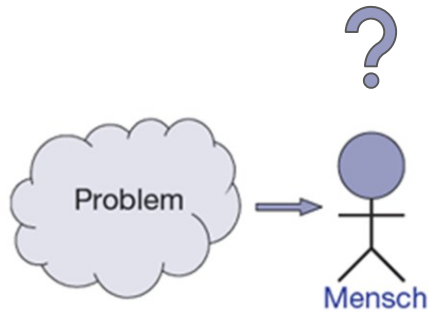
*Kids Make Nutritious Snacks.
Red Tape Holds Up New Bridge.
Police Squad Helps Dog Bite Victim.
Local High School Dropouts Cut in Half.*
[real US newspaper headlines, compiled by Rich Pattis]

- **Höhere Programmiersprachen:**
Akzeptabler Kompromiss.

```
for (int t = 0; t < 2000; t++)
{
    a[0] = a[11] ^ a[9];
    System.out.print(a[0]);
    for (int i = 11; i > 0; i--)
        a[i] = a[i-1];
}
```

12

Wo fangen wir an?



Quelle: vgl. Pomberger/Dobler, 2008

13

Wie finde ich die Programmieraufgabe?

Zentrale Teilaufgabe ist das Verstehen der Kundenwünsche.

Das Verfahren zum Finden der Wünsche heißt **Anforderungsanalyse**:

- Regel: garbage in, garbage out
- systematische Analyse der Kundenherausforderungen
- Strukturierung der gewünschten Funktionalität in Hauptaufgaben
- Klärung der relevanten Informationen zur Modellierung
- Klärung vieler Randbedingungen
- Mehr im nächsten Semester in der Vorlesung Software Engineering

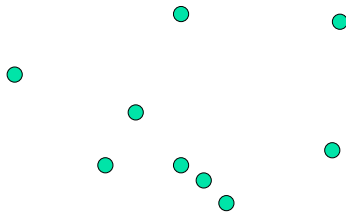
Für diese Veranstaltung werden die Aufgabenstellungen als klar gegeben angenommen (große Vereinfachung).

14

Problemstellungen

Typische Aufgabenstellung:

- Berechne die kürzeste Rundreise durch n Städte (Travelling Salesman Problem)



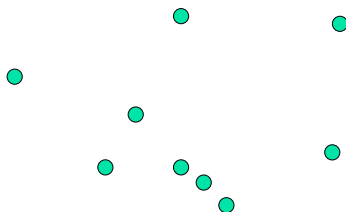
15

Problemstellungen

Typische Aufgabenstellung:

- Berechne die kürzeste Rundreise durch n Städte

Optimierungsproblem

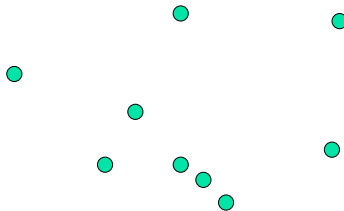


16

Problemstellungen

Optimierungsprobleme (informal):

- Zulässigkeitsbedingung (Lösung ist eine Rundreise)
- Zielfunktion (Länge der Tour)
- Aufgabe: Find beste Lösung!

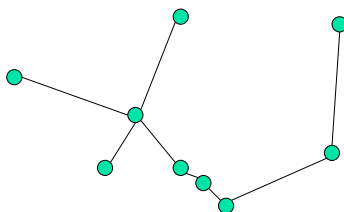


17

Problemstellungen

Optimierungsprobleme (informal):

- Zulässigkeitsbedingung (Lösung ist eine Rundreise)
- Zielfunktion (Länge der Tour)
- Aufgabe: Find beste Lösung



**Keine zulässige Lösung,
da keine Rundreise!**

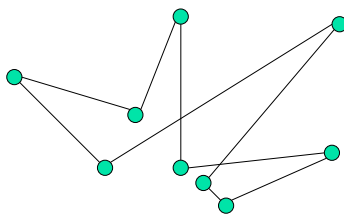
Kosten (Summe der Kanten-
längen) aber kleiner als bei
der besten Rundreise

18

Problemstellungen

Optimierungsprobleme (informal):

- Zulässigkeitsbedingung (Lösung ist eine Rundreise)
- Zielfunktion (Länge der Tour)
- Aufgabe: Find beste Lösung



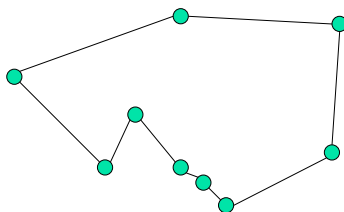
Lösung ist zulässig,
aber nicht optimal

19

Problemstellungen

Optimierungsprobleme (informal):

- Zulässigkeitsbedingung (Lösung ist eine Rundreise)
- Zielfunktion (Länge der Tour)
- Aufgabe: Find beste Lösung



Optimale zulässige
Lösung

20

Problemstellungen

Typische Aufgabenstellung:

- Entscheide, ob eine Zahl prim ist
 - 2 → prim
 - 17 → prim
 - 99 → nicht prim
-

21

Problemstellungen

Typische Aufgabenstellung:

- Entscheide, ob eine Zahl prim ist

Entscheidungsproblem

- 2 → prim
 - 17 → prim
 - 99 → nicht prim
-

22

Problemstellungen

Entscheidungsprobleme:

- Eigenschaft (Primzahl)
 - Aufgabe:
 - Akzeptieren, wenn Eingabe die Eigenschaft besitzt
 - Zurückweisen, sonst
 - Ausgabe: 1 (akzeptieren) oder 0 (zurückweisen)
-

23

Problemstellungen

Typische Aufgabenstellung:

- Sortiere Folge von n Zahlen

Eingabe:

- 9, 3, 2, 15, 17, 8

Ausgabe:

- 2, 3, 8, 9, 15, 17
-

24

Problemstellungen

Typische Aufgabenstellung:

- Sortiere Folge von n Zahlen

Ist das eine neue Art von Problem?

Eingabe:

- 9, 3, 2, 15, 17, 8

Ausgabe:

- 2, 3, 8, 9, 15, 17
-

25

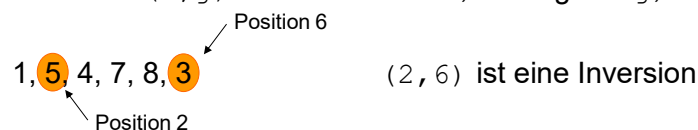
Problemstellungen

Typische Aufgabenstellung:

- Sortiere Folge von n Zahlen

Inversion:

- Das Paar (i, j) ist eine Inversion, wenn gilt $i < j$, aber $x_i > x_j$



Neue Problemformulierung:

- Finde die Reihenfolge der Zahlen mit der kleinsten Anzahl Inversionen
-

26

Problemstellungen

Neue Problemformulierung:

- Finde die Reihenfolge der Zahlen mit der kleinsten Anzahl Inversionen

Optimierungsproblem

Annahme (Lemma):

Eine Reihenfolge ohne Inversionen ist aufsteigend sortiert.

27

Problemstellungen

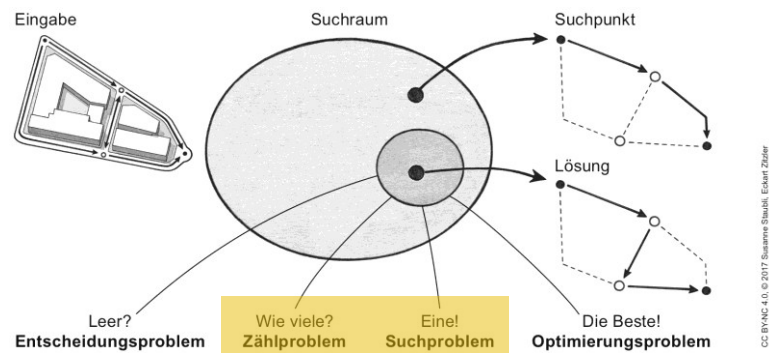
Erkenntnis:

- **Durch geschickte Umformulierung kann man die meisten algorithmischen Probleme als Entscheidungs- oder Optimierungsprobleme formulieren.**
-

28

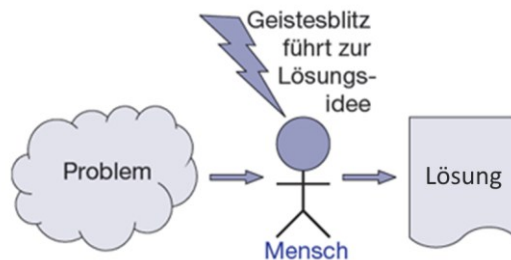
Problemstellungen

- Sind das alle Problemarten? **NEIN!**



29

Wir haben eine Lösung! - Alles gut?

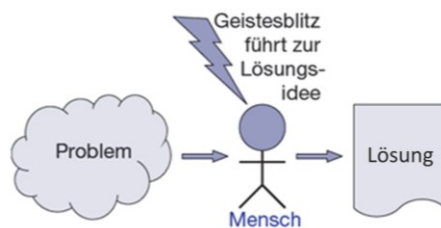


Quelle: vgl. Pomberger/Dobler, 2008

30

Wir haben eine Lösung! - Das nächste Problem!

Wie sagen es wir dem Computer?



© iStockphoto.com

Quelle: vgl. Pomberger/Dobler, 2008

31

Wie fange ich die Programmierung an?

- zunächst klären, ob Aufgabenstellung wirklich verstanden
- typische Entwicklungsschritte:
 - Aufteilung in aufeinander aufbauende Teilaufgaben
 - für jede Teilaufgabe das erwünschte typische Verhalten realisieren (z. B. Daten eines neuen Studierenden eintragen und speichern)
 - über möglichen Alternativen beim Verhalten nachdenken und diese schrittweise einbauen (z. B. Abbruch der Dateneintragung, Fehler beim Speichern)
- Grundlage ist dabei ein Modell der gewünschten Daten, die für die Teilaufgabe benötigt werden
- Bei Datenmodellen spricht man oft von Objekten (Individuen mit konkreten Eigenschaften)

32

Die Semantische Lücke

- unterschiedliche Sprache von Mensch und Maschine
- Komplexitätsabstand zwischen
 - Anforderungen einer Anwendung (Nutzer-Maschine)
 - umsetzbare Möglichkeiten (Basismaschine)

Unsere Weltsicht:

Dinge + Handlungen



Begriffswelt der Informatik:

Daten + Algorithmen

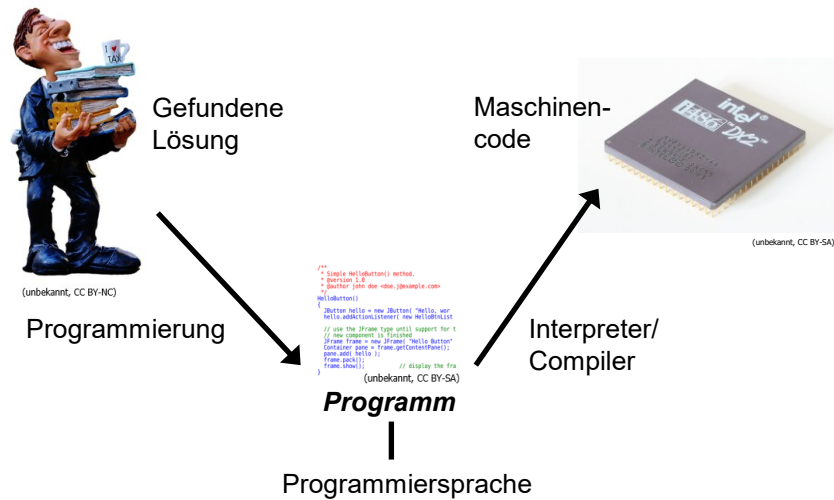
33

Syntax und Semantik von Programmiersprachen

- **Syntax** legt fest,
 - welche Sprachelemente und -konstrukte es gibt und
 - wie mit ihrer Hilfe korrekte Sätze in der Sprache formuliert werden können
 - Syntax = Menge von Regeln, die die Struktur von Programmen bestimmen
- **Semantik** einer Programmiersprache
 - legt die Bedeutung syntaktisch korrekter Sätze fest
 - legt fest, welche Wirkung jedes Sprachelement oder Sprachkonstrukt im Programmablauf hervorruft
 - Semantik = Menge von Verhaltensregeln, die die Funktionsweise von Programmen bestimmen

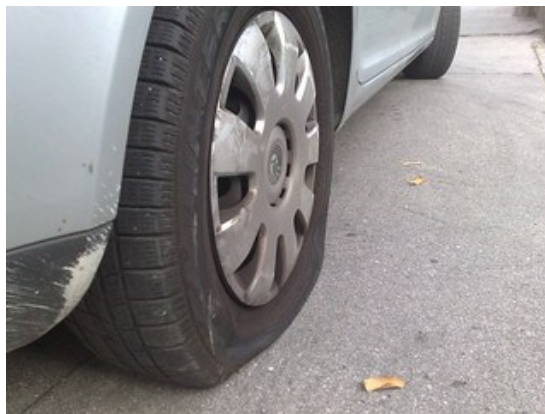
34

Unsere Aufgabe



35

Ein (relativ alltäglicher) Algorithmus



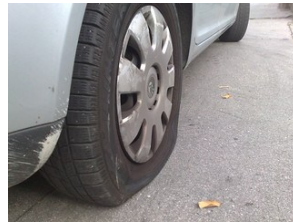
Dieses Foto ist lizenziert gemäß CC BY-SA-NC

36

Ein (relativ alltäglicher) Algorithmus

Beispiel: Auswechseln eines Reifens am Auto

- (1) Löse die Radmuttern.
- (2) Hebe den Wagen an.
- (3) Schraube die Radmuttern ab.
- (4) Tausche das Rad gegen ein anderes Rad aus.
- (5) Schraube die Radmuttern an.
- (6) Setze den Wagen ab.
- (7) Ziehe die Radmuttern fest.



Dieses Foto ist lizenziert gemäß CC BY-SA-NC

37

Bestandteile eines Algorithmus

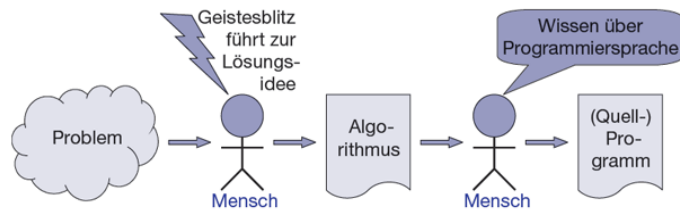
- **Objekte**, auf die eine Wirkung ausgeübt werden soll.
(Diese können sowohl abstrakter als auch konkreter Natur sein)
- **Handlungen**, die - in einer bestimmten Reihenfolge ausgeführt - auf gewünschte Weise auf die Objekte einwirken und **Zustandsänderungen** bewirken.



38

Struktur ist immer von Vorteil

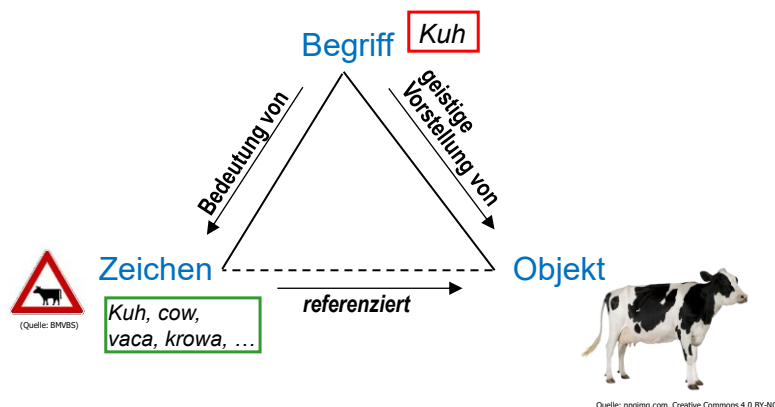
Ein **Algorithmus** ist eine eindeutige Handlungsvorschrift zur Lösung von Instanzen eines Problems in endlich vielen Schritten.



Quelle: vgl. Pomberger/Dobler, 2008

39

Problem: Schwierige Begrifflichkeiten



Ein Computer ist kein Mensch! **Formalisierung notwendig!**

40

Die Mathematik bietet klare Begrifflichkeiten!

Durch Schulbildung allgemeinverständliche formale Darstellung.

Beispiel: Lösung quadratischer Gleichung

$$x^2 + 8x + 7 = 0$$

$$x_{1,2} = -\frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

41

Praxisbeispiel (1/3)

$$x_{1,2} = -\frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

1. Lies die Zahlen p und q

2. Berechne die Zahl d:

$$d = \sqrt{\frac{p^2}{4} - q}$$

3. Berechne die Zahl x_1 :

$$x_1 = -\frac{p}{2} + d$$

4. Berechne die Zahl x_2 :

$$x_2 = -\frac{p}{2} - d$$

5. Schreibe x_1 und x_2 auf

42

Praxisbeispiel (2/3)

$$x_{1,2} = -\frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

1. Lies die Zahlen p und q
2. Berechne $p/2$ und nenne das Ergebnis a
3. Berechne a^2 und nenne das Ergebnis b
4. Berechne $b - q$ und nenne das Ergebnis c
5. Berechne $\sqrt{c}$ und nenne das Ergebnis d
6. Berechne $-a$ und nenne das Ergebnis e
7. Berechne $e + d$ und nenne das Ergebnis x_1
8. Berechne $e - d$ und nenne das Ergebnis x_2
9. Schreibe x_1 und x_2 auf

43

Praxisbeispiel (3/3)

$$x_{1,2} = -\frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

```
int p = 8;           // Schritt 1
int q = 7;
double a = p/2.0;    // Schritt 2
double b = a*a;      // Schritt 3
double c = b - q;    // Schritt 4
double d = Math.Sqrt(c); // Schritt 5
double e = - a;      // Schritt 6
double x1= e + d;    // Schritt 7
double x2= e - d;    // Schritt 8

// Schritt 9
Console.WriteLine("x1= "+x1+", x2= "+x2);
```

44

Engerer Algorithmusbegriff in der Informatik

- (Halb-) formale Darstellung für Entwurf und Bearbeitung von Algorithmen.
- Einzelne **Handlungen** sind „computerverständlich“ formuliert.
- „Computerverständliche“ Darstellung beteiligter Objekte durch Datentypen und -strukturen.
- **Objekte** werden dargestellt, indem man ihre Eigenschaften durch Daten beschreibt, deren **Zustand** sich ändern kann.

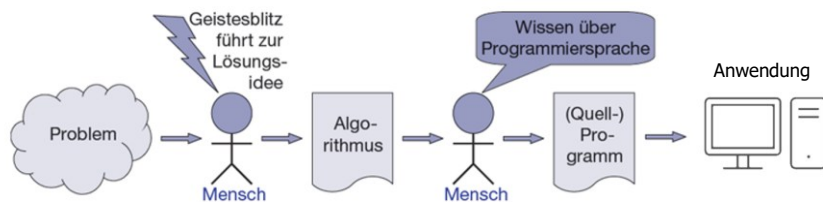
Beispiel:

Person wird durch Namen, Alter, Wohnort usw. beschrieben.

45

Vom Problem zum Algorithmus – Schrittweises Vorgehen

Ausgangspunkt: gegebenes Problem

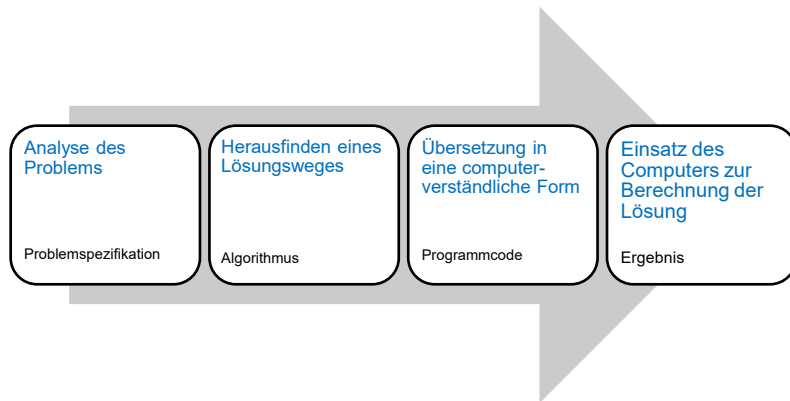


Quelle: vgl. Pomberger/Dobler, 2008

46

Vom Problem zum Algorithmus – Schrittweises Vorgehen

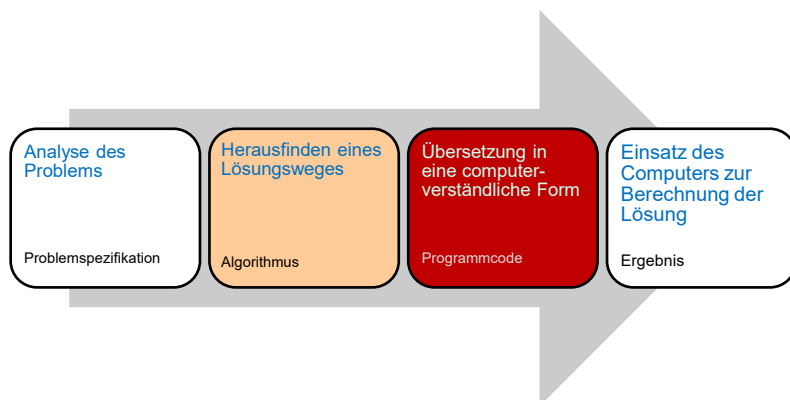
Ausgangspunkt: gegebenes Problem



47

Unsere Schwerpunkte in diesem Semester

Ausgangspunkt: gegebenes Problem



48